Exercícios: Balanceamento de Árvores Binárias

AEDS 2 · PUC Minas · baseado no PDF do professor "Balanceamento de Árvores Binárias (Rotação)"

> **Como usar:** a Parte 1 tem os exercícios que o professor propôs no PDF; a Parte 2 transforma os exemplos dos slides em exercícios. Tente tudo antes de olhar o **gabarito**, que está no final.
> **Convenção do professor:** FB(nó) = altura(direita) − altura(esquerda); subárvore vazia tem altura −1; folha tem altura 0. AVL: todo FB ∈ {−1, 0, +1}.

Parte 1 · Exercícios propostos pelo professor

Exercício 1 (slide "Balanceamento de Árvores")

Qual é a vantagem de uma árvore balanceada? Responda falando do custo de pesquisa, inserção e remoção no pior caso.

Exercício 2 (slide "Implementação da Rotação à Direita")

O professor deu a rotação à esquerda pronta:

```
No rotacionarEsq(No no) {
    No noDir = no.dir;
    No noDirEsq = noDir.esq;

    noDir.esq = no;
    no.dir = noDirEsq;

    return noDir;
}
```

Implemente o método `No rotacionarDir(No no)`, que faz a rotação simples à direita (avô e pai pendem à esquerda: FB(avô) = −2 e FB(pai) = −1).

Exercício 3 (Exercício Resolvido 1)

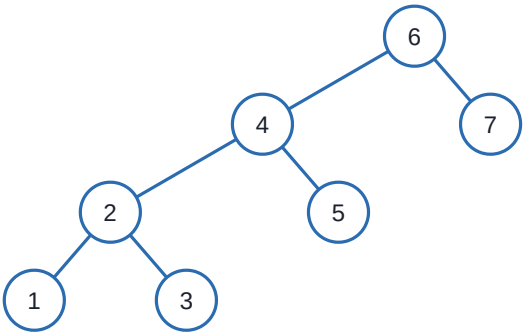
No código da árvore binária, faça um método que leia três números inteiros e os insira na árvore. Se essa árvore estiver com **3 níveis**, efetue uma das 4 rotações apresentadas nesta unidade.

a. Quantas ordens de inserção diferentes existem para 3 elementos distintos (ex.: 1, 2, 3)? Liste todas.
b. Quantas **árvores diferentes** essas ordens geram? Quais ordens geram a mesma árvore?
c. Desenhe cada árvore possível e diga, para cada uma, se já está balanceada ou qual rotação aplicar.
d. Complete os `/*****/` do método abaixo:

```
void balancear() {
    if (raiz.esq != null && raiz.dir != null) {   // casos [2,1,3] e [2,3,1]
        /**************************/
    } else if (raiz.dir != null) {
        if (raiz.dir.dir != null) {                // caso [1,2,3]
            /**************************/
        } else {                                   // caso [1,3,2]
            /**************************/
        }
    } else {                                       // if (raiz.esq != null)
        if (raiz.esq.dir != null) {                // caso [3,1,2]
            /**************************/
        } else {                                   // caso [3,2,1]
            /**************************/
        }
    }
}
```

Exercício 4 (Exercício Resolvido 2)

Leia e insira os números em uma árvore binária **não balanceada** para que ela fique como a figura abaixo. Em seguida, execute o caminhar **pré** e o **central**. Finalmente, efetue uma **rotação à direita no nó avô** para balancear a árvore e execute o caminhar pré e o central novamente.



Avô = 6, Pai = 4, Filho = 2, A = 1, B = 3, C = 5, D = 7

a. Em que ordem os números devem ser inseridos?
b. Caminhar pré e central **antes** da rotação.
c. Calcule FB(6) e FB(4) e confirme que a rotação certa é a simples à direita.
d. Desenhe a árvore depois de `raiz = rotacionarDir(raiz);`
e. Caminhar pré e central **depois**. O que mudou e o que não mudou? Por quê?

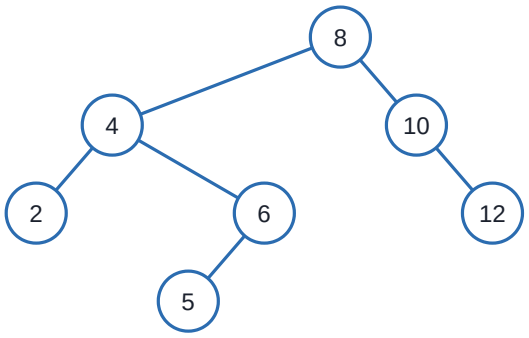
Exercício 5 (slide final "Balanceamento de Árvores")

Qual é o custo para se manter uma árvore **perfeitamente** balanceada? Por que árvores como a AVL aceitam ficar apenas **praticamente** balanceadas?

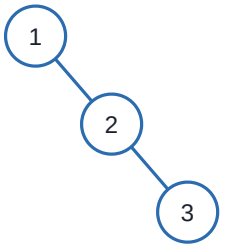
Parte 2 · Exemplos dos slides transformados em exercícios

Exercício 6 · Calcular o FB (slide "Fator de Balanceamento")

Calcule o FB de cada nó das duas árvores. Algum nó está desbalanceado?



Árvore I



Árvore II

Exercício 7 · Degenerada × balanceada (slides "Por que Balancear?" e "Crescimento da Altura")

a. Desenhe a ABB obtida ao inserir **1, 2, 3, 4, 5, 6, 7** e a obtida ao inserir **4, 2, 6, 1, 3, 5, 7**. Qual a altura de cada uma?
b. Mostre que as alturas valem $n - 1$ e $\lfloor \lg n \rfloor$, respectivamente.
c. Com $n = 1.000.000$, quantas comparações faz a pesquisa no pior caso em cada árvore? Quantas vezes a balanceada é mais rápida?

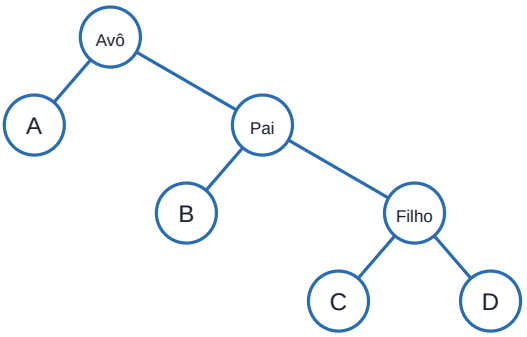
Exercício 8 · Qual rotação? (slide "Qual Rotação Aplicar?")

Complete a tabela sem consultar o material:

FB(avô)	FB(pai)	Situação (para onde pendem)	Rotação
+2	+1		
−2	−1		
+2	−1		
−2	+1		

Responda também: (a) quem é o "pai" do avô desbalanceado? (b) Quando acontece FB(pai) = 0 e como se resolve?

Exercício 9 · Rotação à esquerda no caso geral (slides "Rotação à Esquerda: Resultado" e "Por que a Rotação é Válida?")

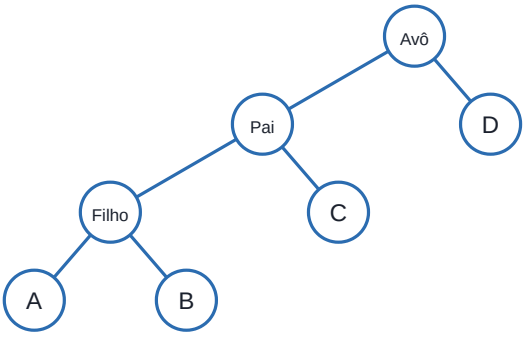


A, B, C, D são subárvores (podem ser vazias)

a. Desenhe o resultado de `rotacionarEsq(avô)`.
b. Escreva o caminhamento central antes e depois.
c. Qual subárvore troca de pai? Por que ela pode virar filha direita do Avô sem quebrar a ABB?
d. Quantos ponteiros são alterados? Qual o custo da rotação? Qual a pré-condição para chamar o método?

Exercício 10 · Rotação à direita no caso geral (slide "Rotação à Direita: Resultado")

Agora a árvore é Avô com filho esquerdo Pai e filho direito D; Pai tem filho esquerdo Filho e filho direito C; Filho tem filhos A e B. Desenhe o resultado de `rotacionarDir(avô)`, escreva o central antes e depois e diga qual subárvore troca de pai.



Exercício 11 · Encontre o erro (slide "Cuidado com o Retorno da Rotação")

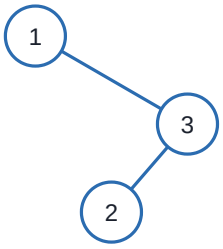
```
void balancear(No pai) {
    rotacionarEsq(pai.dir);
}
```

a. O que está errado? Para onde `pai.dir` fica apontando depois da chamada? Quais nós ficam inacessíveis?
b. Corrija o código. Como fica a chamada se o nó rotacionado for a própria raiz?
c. Que linha da inserção recursiva segue o mesmo padrão?

Exercício 12 · Rotações são inversas (slide "Rotações São Inversas")

Seja x com filho esquerdo A e filho direito y, e y com filhos B e C. Desenhe `rotacionarEsq(x)` e, em seguida, aplique `rotacionarDir` na nova raiz. O que você obtém?

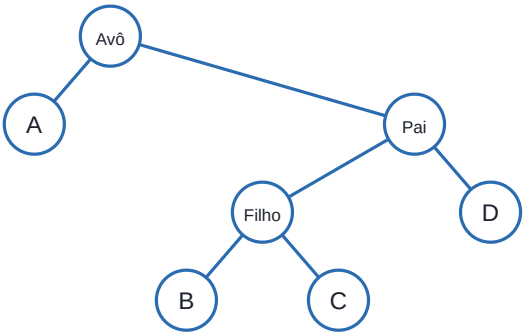
Exercício 13 · Limite da rotação simples (slide "Limite da Rotação Simples")



inserção 1, 3, 2

a. Calcule os FB e identifique avô, pai e filho.
b. Aplique (errado de propósito) `rotacionarEsq(1)`. Desenhe o resultado e calcule os FB. Resolveu?
c. Qual a rotação correta? Faça-a em dois passos e desenhe cada passo.

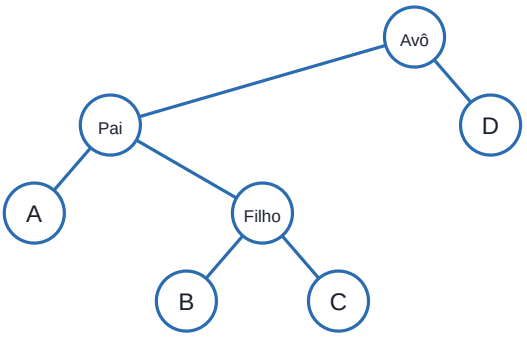
Exercício 14 · Dupla direita-esquerda no caso geral (slides "Rotação Dupla Direita-Esquerda" e "Caso Geral")



FB(avô) = +2, FB(pai) = −1

a. Escreva `No rotacionarDirEsq(No no)` reaproveitando as rotações simples.
b. Desenhe o resultado. Quem vira a raiz?
c. Para onde vão B e C?
d. Escreva o central antes e depois.

Exercício 15 · Dupla esquerda-direita no caso geral (slides "Rotação Dupla Esquerda-Direita" e "Caso Geral")



FB(avô) = −2, FB(pai) = +1

a. Escreva `No rotacionarEsqDir(No no)`.
b. Desenhe o resultado e diga para onde vão B e C.
c. Escreva o central antes e depois.

Gabarito

> Só confira depois de tentar! As respostas marcadas como **(professor)** estão no PDF; as outras são resoluções passo a passo feitas a partir do conteúdo dos slides.

Exercício 1 (professor)

Eficiência em pesquisa, inserção e remoção: **O(lg n)** em vez de **O(n)** no pior caso. O custo dessas operações é proporcional à **altura** da árvore; balancear mantém a altura perto de lg n. Uma árvore começa vazia (balanceada), mas pode se desbalancear após inserções e remoções; por isso é preciso rebalancear.

Exercício 2 (professor)

```
No rotacionarDir(No no) {
    No noEsq = no.esq;        // o filho esquerdo (Pai) vai subir
    No noEsqDir = noEsq.dir;  // guarda C antes de perdê-la

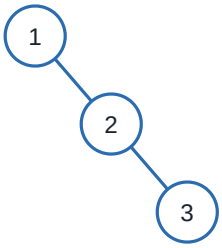
    noEsq.dir = no;           // o Avô desce e vira filho direito do Pai
    no.esq = noEsqDir;        // C é pendurada à esquerda do Avô

    return noEsq;             // o Pai é a nova raiz da subárvore
}
```

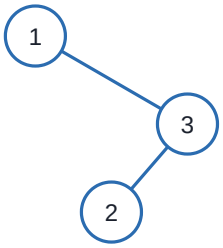
Truque: é o espelho exato de `rotacionarEsq`: troque todo `esq` por `dir` e vice-versa.

Exercício 3 (professor)

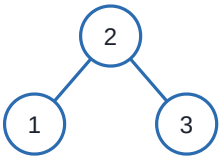
a. São 3 × 2 × 1 = **3! = 6** ordens: [1,2,3], [1,3,2], [2,1,3], [2,3,1], [3,1,2], [3,2,1].
b. **5 árvores**: [2,1,3] e [2,3,1] geram a mesma árvore (o 2 vira raiz e cada um dos outros vai para o seu lado, não importa a ordem). Esse número 5 é o número de Catalan C_3.
c. Árvores possíveis:



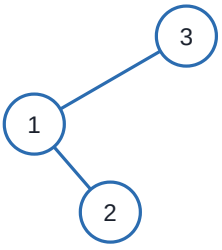
[1,2,3] → simples à esq.



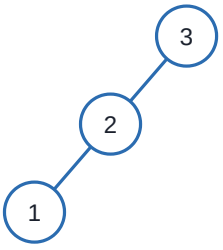
[1,3,2] → dupla dir-esq



[2,1,3] e [2,3,1] → já balanceada



[3,1,2] → dupla esq-dir



[3,2,1] → simples à dir.

Só a árvore com raiz 2 tem 2 níveis. As outras 4 têm 3 níveis (uma "corrente"), e cada uma corresponde a uma das 4 rotações.

d) Código completo:

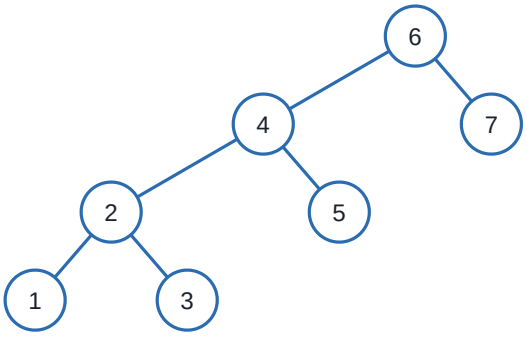
```
void balancear() {
    if (raiz.esq != null && raiz.dir != null) {   // casos [2,1,3] e [2,3,1]
        System.out.println("Árvore balanceada");
    } else if (raiz.dir != null) {
        if (raiz.dir.dir != null) {                // [1,2,3]: reta à direita
            raiz = rotacionarEsq(raiz);
        } else {                                   // [1,3,2]: joelho dir-esq
            raiz = rotacionarDirEsq(raiz);
        }
    } else {                                       // if (raiz.esq != null)
        if (raiz.esq.dir != null) {                // [3,1,2]: joelho esq-dir
            raiz = rotacionarEsqDir(raiz);
        } else {                                   // [3,2,1]: reta à esquerda
            raiz = rotacionarDir(raiz);
        }
    }
}
```

Repare que toda chamada faz `raiz = ...`: a rotação muda quem é a raiz. A leitura dos três números pode ser feita assim (só a parte de ler e inserir; o PDF mostra apenas o `balancear`):

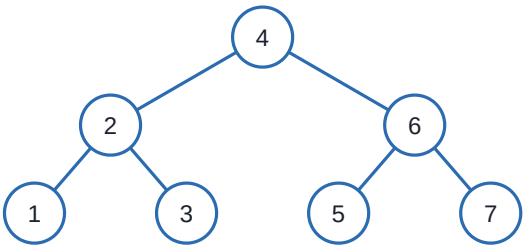
```
void lerInserirBalancear() throws Exception {
    Scanner sc = new Scanner(System.in);  // leitor do teclado
    for (int k = 0; k < 3; k++) {         // repete 3 vezes
        inserir(sc.nextInt());            // lê um inteiro e insere na árvore
    }
    balancear();                          // decide se e como rotacionar
}
```

Exercício 4 (professor)

a. Uma ordem que funciona é **6, 4, 7, 2, 5, 1, 3** (cada pai precisa ser inserido antes dos filhos; por exemplo, por níveis).
b. Antes: **pré** = 6, 4, 2, 1, 3, 5, 7 · **central** = 1, 2, 3, 4, 5, 6, 7.
c. Alturas: 1, 3, 5, 7 são folhas (0); h(2) = 1; h(4) = 1 + max(1, 0) = 2. FB(4) = h(5) − h(2) = 0 − 1 = **−1**; FB(6) = h(7) − h(4) = 0 − 2 = **−2**. Avô (−2) e pai (−1) pendem para a esquerda → **simples à direita**.
d. Depois de `raiz = rotacionarDir(raiz);` o 4 sobe e o 5 (subárvore C) muda de pai, indo para a esquerda do 6:



antes



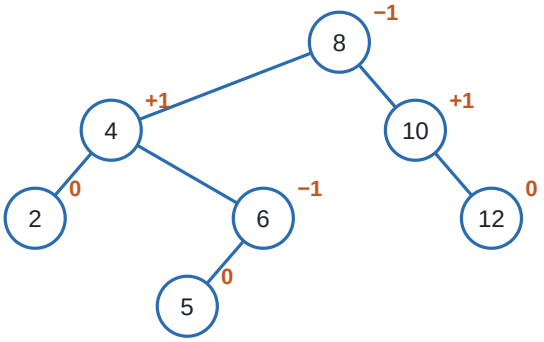
depois (todos os FB = 0)

e) Depois: **pré** = 4, 2, 1, 3, 6, 5, 7 · **central** = 1, 2, 3, 4, 5, 6, 7. O pré mudou (a raiz mudou), mas o **central não muda**: a rotação preserva a ordem, então continua sendo uma ABB.

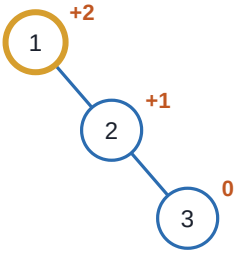
Exercício 5 (professor)

Manter a árvore **perfeitamente** balanceada pode exigir reorganizar grande parte dela a cada inserção: custo **O(n)** por operação, o que acaba com a vantagem. Na prática não há "muita" diferença entre uma árvore balanceada e uma **praticamente** balanceada; por isso a AVL (e a árvore Bicolor, citada no slide) aceitam um pequeno desequilíbrio e continuam com altura O(lg n): AVL ≈ 1,44·lg n; Bicolor ≤ 2·lg(n + 1). E cada rotação custa só O(1).

Exercício 6 (professor)



Árvore I: todos os FB ∈ {−1, 0, +1}

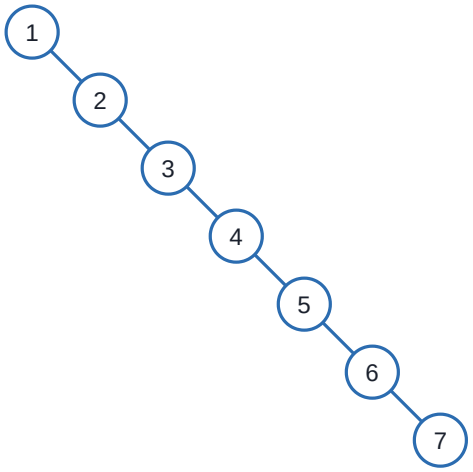


Árvore II: FB(1) = +2, desbalanceado

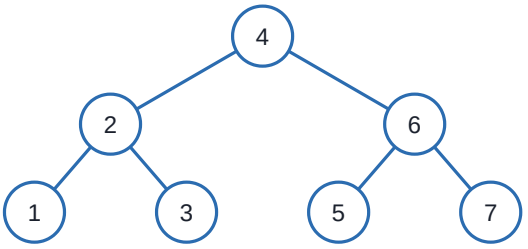
Nó (Árvore I)	h(esq)	h(dir)	FB	altura
2	−1	−1	0	0
5	−1	−1	0	0
12	−1	−1	0	0
6	0	−1	−1	1
10	−1	0	+1	1
4	0	1	+1	2
8	2	1	−1	3

Árvore II: h(3) = 0 → FB(3) = 0; h(2) = 1, FB(2) = 0 − (−1) = +1; FB(1) = 1 − (−1) = **+2**. É o caso de rotação simples à esquerda (avô +2, pai +1).

Exercício 7 (professor)



1..7 em ordem: altura 6 = n − 1



4, 2, 6, 1, 3, 5, 7: altura 2 = ⌊lg 7⌋

b) Na degenerada cada nó tem um único filho, então há n níveis e n − 1 arestas: altura 7 − 1 = 6. Na balanceada, ⌊lg 7⌋ = ⌊2,81⌋ = 2.

c) Comparações no pior caso = número de níveis. Degenerada: **1.000.000**. Balanceada: ⌊lg 1.000.000⌋ + 1 = 19 + 1 = **20** (2^{19} = 524.288 ≤ 10^6 < 2^{20}). 1.000.000 / 20 = **≈ 50.000 vezes menos**.

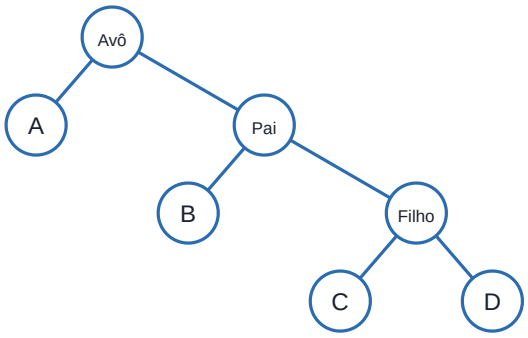
Exercício 8 (professor)

FB(avô)	FB(pai)	Situação	Rotação
+2	+1	avô e pai pendem para a direita	simples à esquerda
−2	−1	avô e pai pendem para a esquerda	simples à direita
+2	−1	avô à direita, pai à esquerda	dupla direita-esquerda
−2	+1	avô à esquerda, pai à direita	dupla esquerda-direita

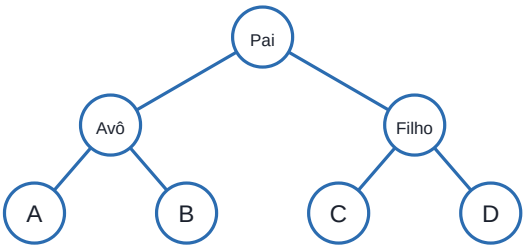
(a) O pai é o filho do avô **no lado mais alto**. (b) FB(pai) = 0 só ocorre na **remoção** e é resolvido com uma rotação **simples**.

Padrão: sinais iguais → simples; sinais opostos → dupla.

Exercício 9 (professor)



antes



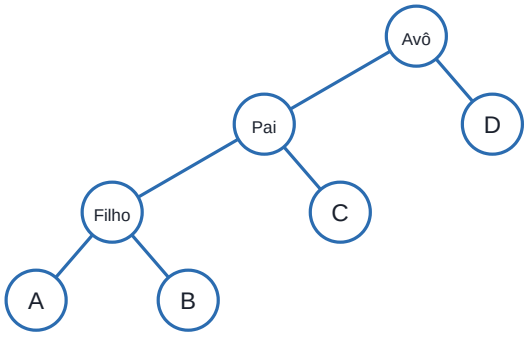
depois de rotacionarEsq(avô)

b) Central antes e depois: **A, Avô, B, Pai, C, Filho, D** (igual).

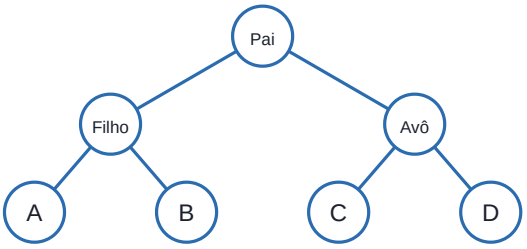
c) A subárvore **B** sai do Pai e vira a subárvore direita do Avô. Pode, porque todos os elementos de B são **maiores que o Avô** (estavam à direita dele) e **menores que o Pai** (estavam à esquerda dele).

d) Só **dois ponteiros** mudam (`noDir.esq` e `no.dir`), qualquer que seja o tamanho das subárvores → **O(1)**. Pré-condição: `no.dir != null` (em `rotacionarDir`, `no.esq != null`).

Exercício 10 (professor)



antes



depois de rotacionarDir(avô)

Central antes e depois: **A, Filho, B, Pai, C, Avô, D**. A subárvore **C** troca de pai: sai do Pai e vira a subárvore **esquerda** do Avô.

Exercício 11 (professor)

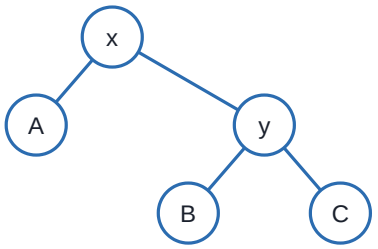
a. `rotacionarEsq` **devolve** a nova raiz da subárvore, mas o retorno foi jogado fora. `pai.dir` continua apontando para o **antigo Avô**, que agora está embaixo. O **Pai** (nova raiz) e o **Filho** (e as subárvores C e D) ficam inacessíveis.

b.
```
void balancear(No pai) {
    pai.dir = rotacionarEsq(pai.dir);   // pendura a nova raiz no lugar certo
}

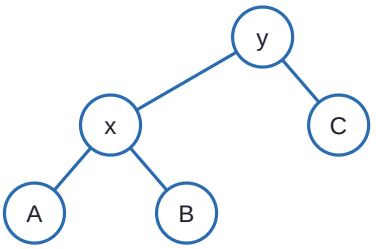
// se o nó rotacionado for a raiz da árvore:
raiz = rotacionarEsq(raiz);
```

c. O mesmo padrão da inserção recursiva: `i.esq = inserir(x, i.esq);`

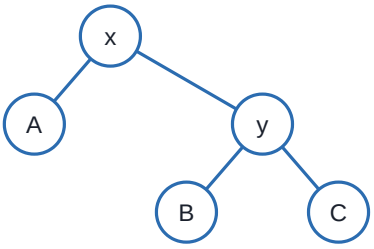
Exercício 12 (professor)



original



rotacionarEsq(x)

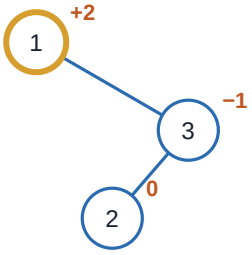


rotacionarDir(y): volta ao original

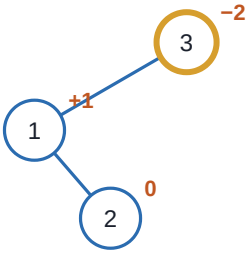
A rotação à esquerda e a rotação à direita **desfazem uma à outra**: `rotacionarDir(rotacionarEsq(x))` devolve a árvore original. Em ambas, B só troca de pai.

Exercício 13 (professor)

a) FB(2) = 0, FB(3) = −1 (só tem filho esquerdo), FB(1) = +2. Avô = 1, pai = 3, filho = 2. Avô pende para a direita e pai para a esquerda.



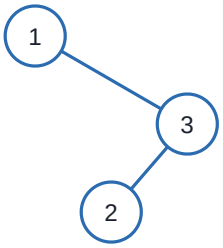
antes



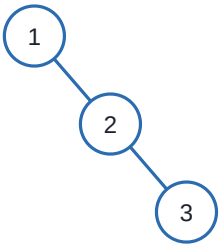
depois de rotacionarEsq(1): continua desbalanceada

b) **Não resolveu**: virou o **espelho** do caso original (agora FB(3) = −2 e FB(1) = +1).

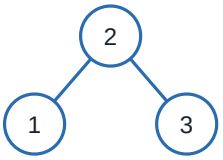
c) Solução: **alinhar o pai** (rotação no pai) e depois **balancear o avô** → rotação **dupla direita-esquerda**:



antes



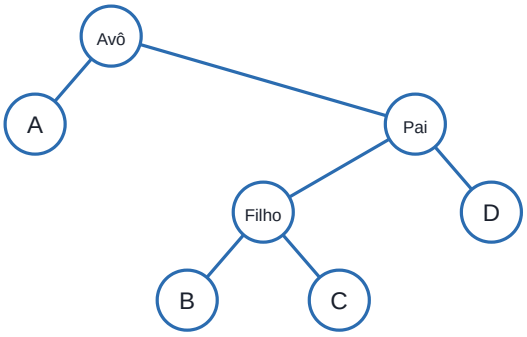
1º: rotacionarDir(3) alinha



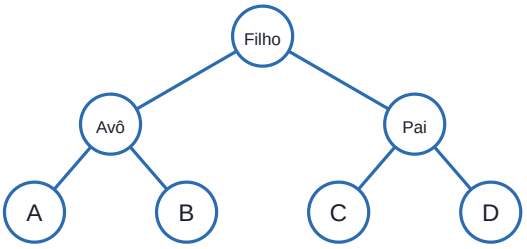
2º: rotacionarEsq(1) balanceia

Exercício 14 (professor)

```
No rotacionarDirEsq(No no) {
    no.dir = rotacionarDir(no.dir);   // 1º: alinha (Pai p/ direita)
    return rotacionarEsq(no);         // 2º: balanceia (Avô p/ esquerda)
}
```



antes



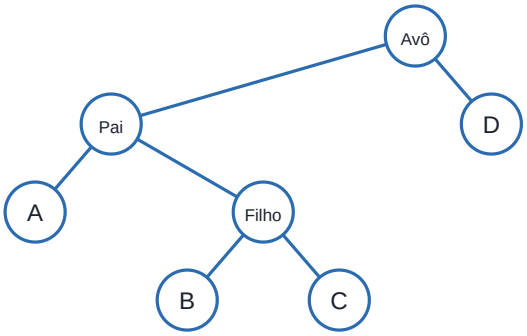
depois de rotacionarDirEsq(avô)

b) O **Filho** (neto do Avô) vira a raiz. c) Os filhos do Filho são repartidos: **B vai para a direita do Avô** e **C para a esquerda do Pai**. d) Central antes e depois: **A, Avô, B, Filho, C, Pai, D**.

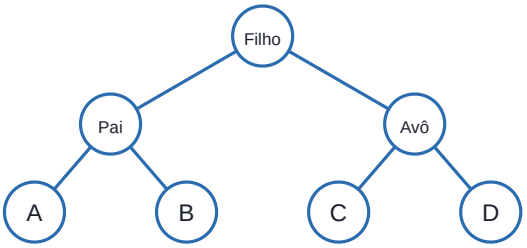
> Esquecer o `no.dir =` na primeira linha é o erro clássico: a rotação do Pai acontece, mas o Avô continua apontando para o Pai antigo.

Exercício 15 (professor)

```
No rotacionarEsqDir(No no) {
    no.esq = rotacionarEsq(no.esq);   // 1º: alinha (Pai p/ esquerda)
    return rotacionarDir(no);         // 2º: balanceia (Avô p/ direita)
}
```



antes



depois de rotacionarEsqDir(avô)

O Filho vira a raiz; **B vai para a direita do Pai** e **C para a esquerda do Avô**. Central antes e depois: **A, Pai, B, Filho, C, Avô, D**.